Leo Alvelais

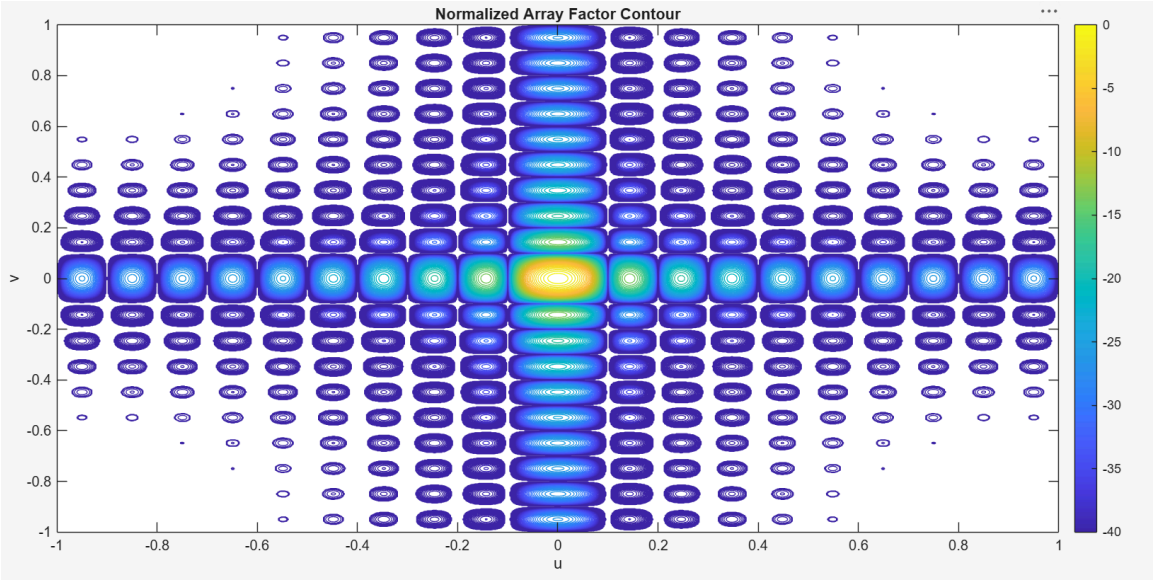
Design of 2.45GHz Patch Antenna Phased Array with Circular Taylor Aperture

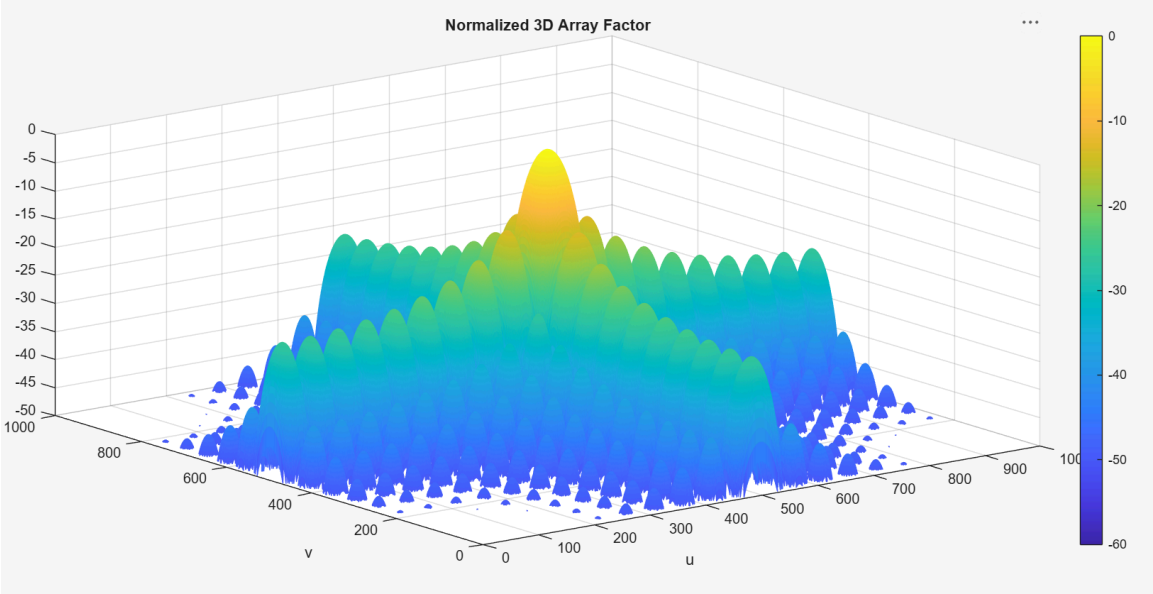
Introduction: This document summarized the design, modelling, and simulation of a 2.4GHz phased array antenna with a circular taylor aperture distribution. A probe fed patch with chamfered corners is chosen for a unit cell. The unit cell elements are positioned along a circular aperture with a diameter of approximately 10 wavelengths at 2.4 GHz. All the relevant theory and mathematics developing the amplitude taper is from [1]. PyAEDT is used to script the taylor amplitude weighting across the array in HFSS.

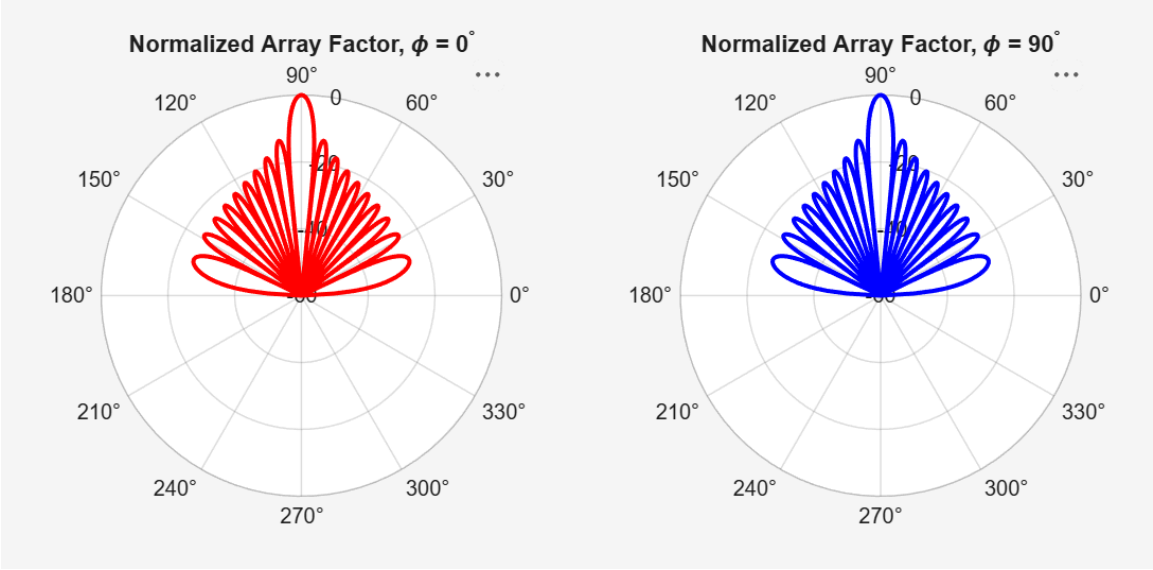
Part 1: Matlab Array Modelling

MATLAB was used to compare uniform square and circular apertures with a circular Taylor distribution. The plots show how the aperture boundary and excitation affect the sidelobes.

Uniform excitation produced a narrow broadside beam. The square boundary formed a grid of sidelobes and nulls in the contour plot. The matching principal-plane cuts are consistent with symmetry about the aperture center.







MATLAB Code

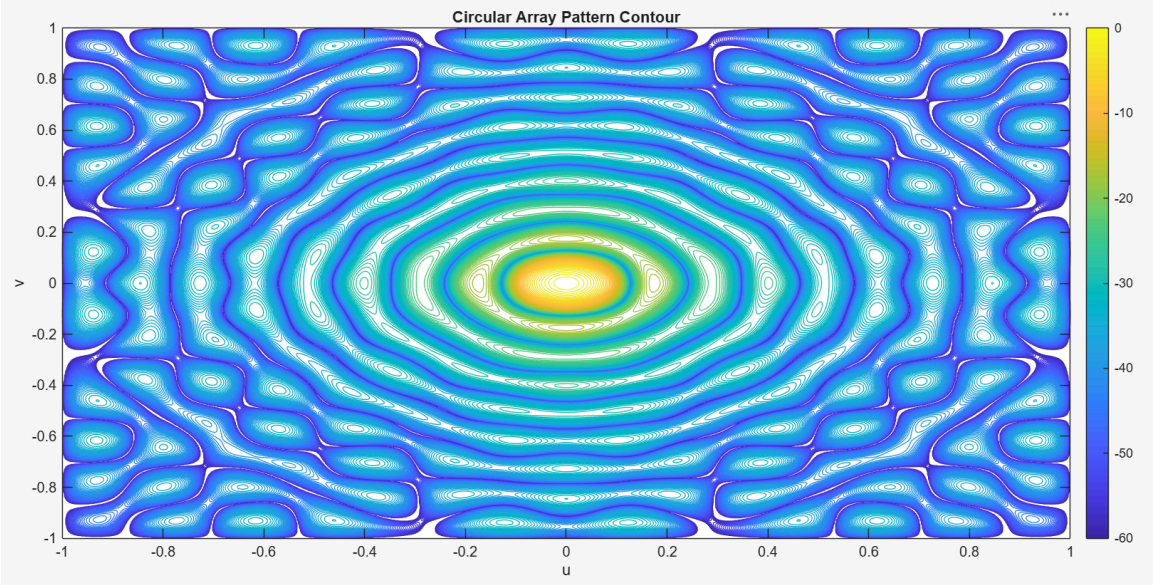
```
%% Circular Aperture
Nx = 20;
Ny = 20;
d = 0.5;
w = ones(Nx, Ny);
pts = 1.001e3;
u = linspace(-1,1, pts);
v = linspace(-1,1, pts);
[uGrid, vGrid] = meshgrid(u,v);
xi = (-(Nx-1)/2:(Nx-1)/2) * d;
yi = (-(Ny-1)/2:(Ny-1)/2) * d;
% cutout elements that aren't within 5lambda from origin
[ElGridx ElGridy] = meshgrid(xi, yi);
```

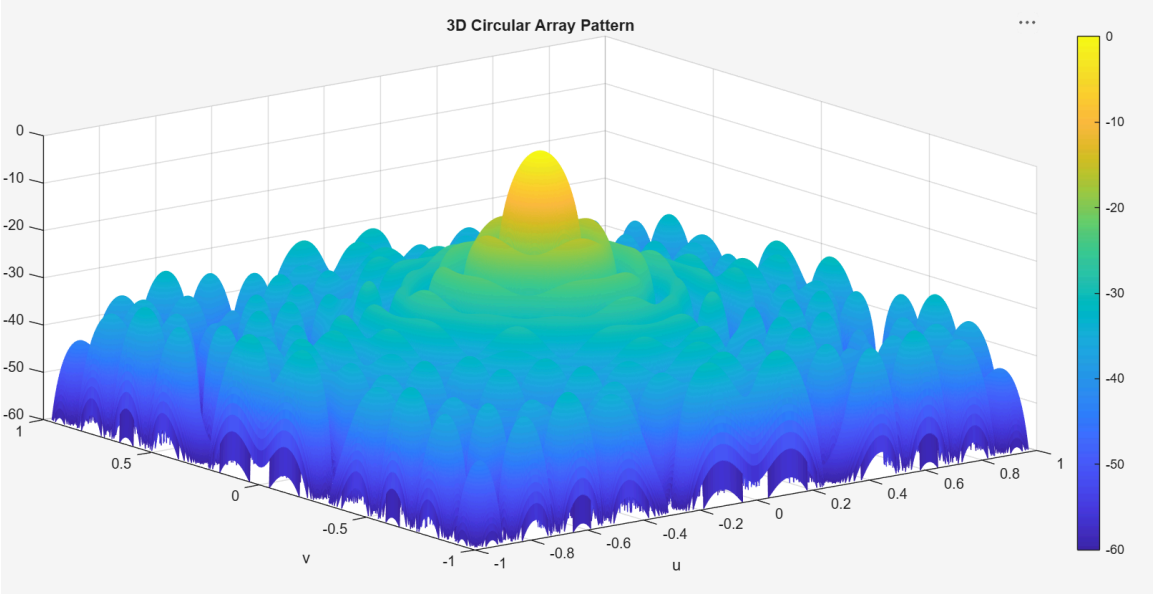
```
w(sqrt( (ElGridx+d/2).^2 + (ElGridy + d/2).^2)>5 ) = 0;
[~,AF_dB, ~] = compAF_UAplanar(pts, Nx, Ny, w,d);
AF_normdB = AF_dB - max(max(AF_dB));
%find elements that need to be cutout
ElNc = find(w == 0);
num_ElNc = size(ElNc, 1);
disp(num_ElNc);
figure(1);
AF_normdB(AF_normdB < -60) = -60;
contour(uGrid, vGrid, AF_normdB, 100);
colorbar;
clim([-60 0]);
xlabel('u');
ylabel('v');
title('Circular Array Pattern Contour');
figure(2);
c = AF_normdB;
c(c<=-60) = NaN;
mesh(uGrid, vGrid, c);
colorbar;
clim([-60 0]);
xlabel('u');
ylabel('v');
title('3D Circular Array Pattern');
%2D Plane Cuts
%2D Plane cuts
figure(3);
u0 = find(u == 0);
v0 = find(v == 0);
tiledlayout(1,2);
nexttile;
theta = acos(u);
rho = AF_normdB(u0, :);
h = polarplot(theta,rho+60, 'LineWidth',2, 'color','r');
haxes = get(h, 'Parent');
haxes.RTickLabel = {'-60','-40','-20','0'};
title('Normalized Array Factor, \phi = 0^\circ');
nexttile;  % degrees for polar plot
rho = AF_normdB(:, v0);
h = polarplot(theta,rho+60, 'LineWidth',2, 'color','b');
haxes = get(h, 'Parent');
haxes.RTickLabel = {'-60','-40','-20','0'};
title('Normalized Array Factor, \phi = 90^\circ');
%FUNCTIONS
function [AF, AF_dB, AFphase] = compAF_UAplanar(pts, Nx, Ny, w,d)
%w should have size mxm
%coordinate transform
u = linspace(-1,1, pts);
v = linspace(-1,1, pts);
[uGrid, vGrid] = meshgrid(u,v);
xi = (-(Nx-1)/2:(Nx-1)/2) * d;
```

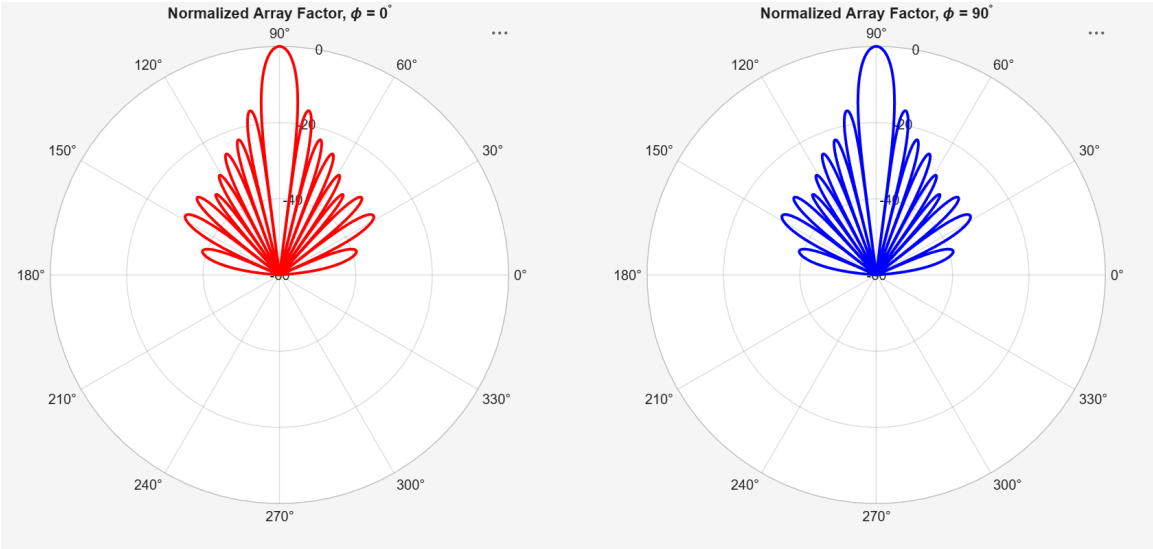
```
yi = (-(Ny-1)/2:(Ny-1)/2) * d;
AF = zeros(pts,pts);
for n = 1:Nx
   for m = 1:Ny
      AF = AF + w(n,m) *exp(1i * 2 * pi * (xi(n) * ...
         uGrid + yi(m) * vGrid ) );
   end
end
AF_dB = 20.* log10(abs(AF));
AFphase = angle(AF);
end
```

Part 2 Uniform Circular Aperture

Removing the corner elements changed the sidelobes from a rectangular pattern to roughly concentric rings. The rings begin to distort when observed farther off of broadside because of the lack of resolution across the aperture. The principal-plane cuts remained similar because the retained aperture is symmetric. The grid still produces departures from an ideal circular boundary.







The total number of elements removed from the square aperture is 124 as calculated by the MATLAB script below.

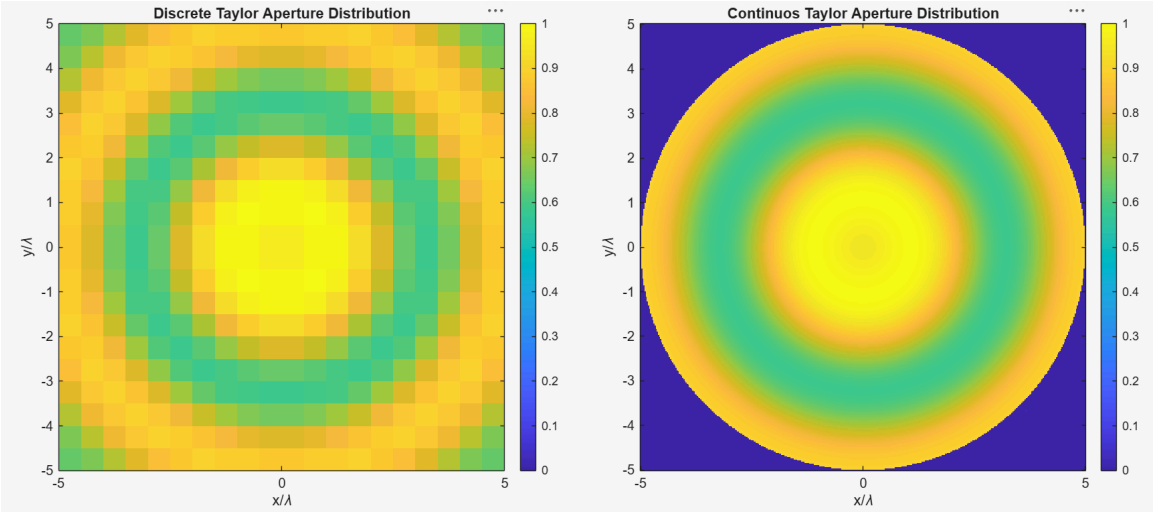
MATLAB Code

```
%% Circular Aperture
Nx = 20;
Ny = 20;
d = 0.5;
w = ones(Nx, Ny);
pts = 1.001e3;
u = linspace(-1,1, pts);
v = linspace(-1,1, pts);
[uGrid, vGrid] = meshgrid(u,v);
xi = (-(Nx-1)/2:(Nx-1)/2) * d;
yi = (-(Ny-1)/2:(Ny-1)/2) * d;
% cutout elements that aren't within 5lambda from origin
[ElGridx ElGridy] = meshgrid(xi, yi);
w(sqrt( (abs(ElGridx)+d/2).^2 + (abs(ElGridy) + d/2).^2)>5 ) = 0;
[~,AF_dB, ~] = compAF_UAplanar(pts, Nx, Ny, w,d);
AF_normdB = AF_dB - max(max(AF_dB));
%find elements that need to be cutout
ElNc = find(w == 0);
num_ElNc = size(ElNc, 1);
disp(num_ElNc);
figure(1);
AF_normdB(AF_normdB < -60) = -60;
contour(uGrid, vGrid, AF_normdB, 100);
colorbar;
clim([-60 0]);
xlabel('u');
ylabel('v');
title('Circular Array Pattern Contour');
figure(2);
c = AF_normdB;
```

```
c(c<=-60) = NaN;
mesh(uGrid, vGrid, c);
colorbar;
clim([-60 0]);
xlabel('u');
ylabel('v');
title('3D Circular Array Pattern');
%2D Plane Cuts
%2D Plane cuts
figure(3);
u0 = find(u == 0);
v0 = find(v == 0);
tiledlayout(1,2);
nexttile;
theta = acos(u);
rho = AF_normdB(u0, :);
h = polarplot(theta,rho+60, 'LineWidth',2, 'color','r');
haxes = get(h, 'Parent');
haxes.RTickLabel = {'-60','-40','-20','0'};
title('Normalized Array Factor, \phi = 0^\circ');
nexttile;  % degrees for polar plot
rho = AF_normdB(:, v0);
h = polarplot(theta,rho+60, 'LineWidth',2, 'color','b');
haxes = get(h, 'Parent');
haxes.RTickLabel = {'-60','-40','-20','0'};
title('Normalized Array Factor, \phi = 90^\circ');
%FUNCTIONS
function [AF, AF_dB, AFphase] = compAF_UAplanar(pts, Nx, Ny, w,d)
%w should have size mxm
%coordinate transform
u = linspace(-1,1, pts);
v = linspace(-1,1, pts);
[uGrid, vGrid] = meshgrid(u,v);
xi = (-(Nx-1)/2:(Nx-1)/2) * d;
yi = (-(Ny-1)/2:(Ny-1)/2) * d;
AF = zeros(pts,pts);
for n = 1:Nx
  for m = 1:Ny
     AF = AF + w(n,m) *exp(1i * 2 * pi * (xi(n) * ...
        uGrid + yi(m) * vGrid ) );
  end
end
AF_dB = 20.* log10(abs(AF));
AFphase = angle(AF);
end
```

Parts 3 and 4 Taylor Aperture Distribution

The continuous distribution was computed from the theory in [1] and sampled at the element centers. The radial variation appears smooth in the continuous map and blocklike on the array grid. The color scale shows normalized excitation amplitude.



MATLAB Code

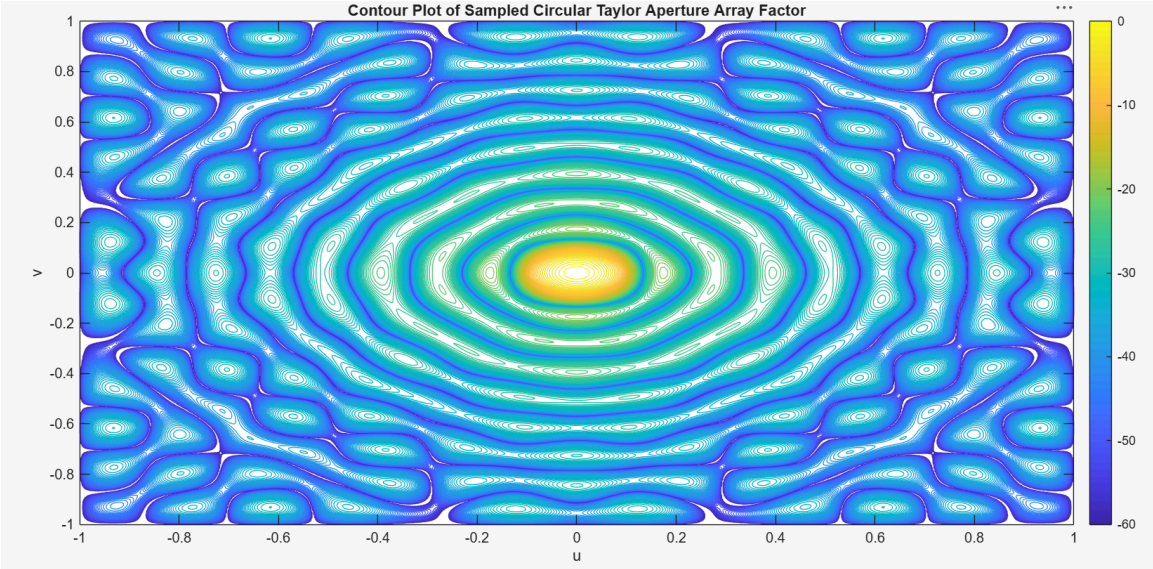
```
%% Circular Aperture
Nx = 20;
Ny = 20;
a = 5; %in lambda
d = 0.5;
SLL = 20;
nbar = 4;
pts = 1.001e3;
u = linspace(-1,1, pts);
v = linspace(-1,1, pts);
[uGrid, vGrid] = meshgrid(u,v);
xi = (-(Nx-1)/2:(Nx-1)/2) * d;
yi = (-(Ny-1)/2:(Ny-1)/2) * d;
% cutout elements that aren't within 5lambda from origin
[ElGridx ElGridy] = meshgrid(xi, yi);
w(sqrt( (abs(ElGridx)+d/2).^2 + (abs(ElGridy) + d/2).^2)>5 ) = 0;
Elx_Nc = ElGridx( sqrt((abs(ElGridx)+d/2).^2 + (abs(ElGridy) + d/2).^2)>5);
Ely_Nc = ElGridy( sqrt(abs((ElGridx+d/2)).^2 + abs((ElGridy + d/2)).^2)>5);
num_ElNc = numel(Elx_Nc);
%compute continuous and discrete aperture distribution
j1 = @(x) besselj(1, x);
mu = zeros(1, nbar);
for i = 1:nbar
   interval = [i*pi, (i + 0.5)*pi];
   mu(i) = fzero(j1, interval);
end
gamma = mu/pi;
A = acosh(10^(SLL/20))/pi;
```

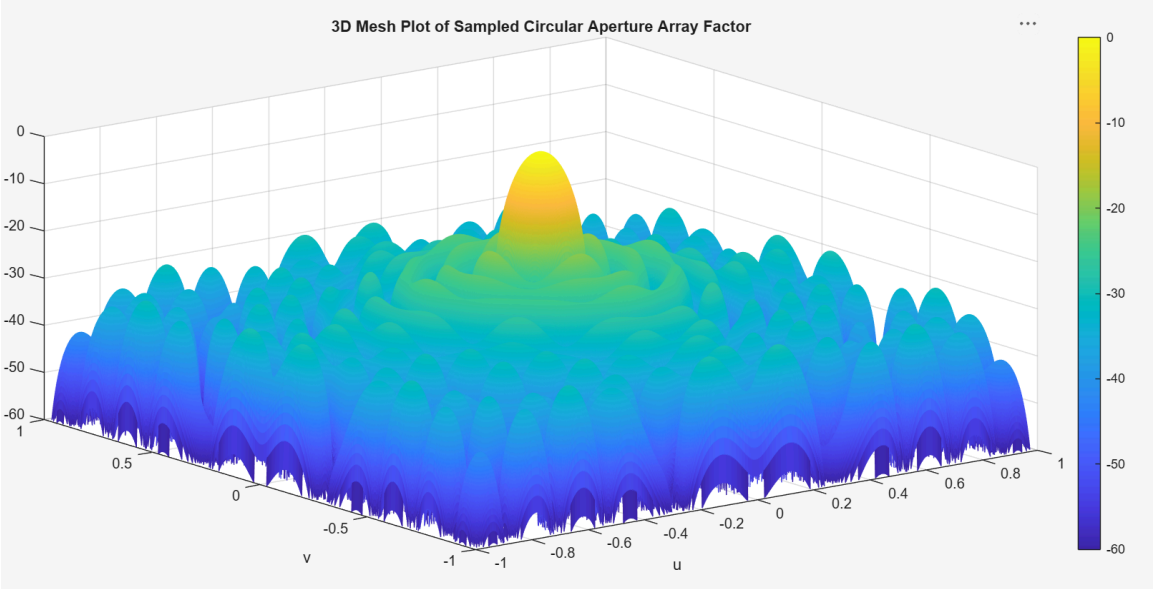
```
% New null locations in lecture notation
n = 1:nbar-1;
un = gamma(nbar) * sqrt( ...
   (A^2 + (n - 0.5).^2) / ...
   (A^2 + (nbar - 0.5)^2));
%discrete distr
[~,w] = circ_TDw(a, gamma, un,nbar, SLL, xi, yi,d);
%cont distr.
xFine = linspace(-a, a, 401);
[Xfine, Yfine] = meshgrid(xFine, xFine);
rhoFine = hypot(Xfine, Yfine);
inside = rhoFine <= a;
G = zeros(size(rhoFine));
G(inside) = Taylor_samp(gamma, un, nbar, ...
   pi*rhoFine(inside)/a);
G = G/max(G(:));
%plot distr.
%disc
figure(1);
tiledlayout(1,2);
nexttile;
imagesc(xi, yi,w);
axis image;
axis xy;
clim([0 1]);
colorbar;
xlabel('x/\lambda');
ylabel('y/\lambda');
title('Discrete Taylor Aperture Distribution');
%cont
nexttile;
imagesc(xFine,xFine, G);
axis image;
axis xy;
clim([0 1]);
colorbar;
xlabel('x/\lambda');
ylabel('y/\lambda');
title('Continuos Taylor Aperture Distribution');
%
%
% [~,AF_dB, ~] = compAF_UAplanar(pts, Nx, Ny, w,d);
%
% AF_normdB = AF_dB - max(max(AF_dB));
%FUNCTIONS
function [g, w] = circ_TDw(a,gamma,un, nbar, SLL, xi, yi,d)
   pts = 1e4;
   rho = linspace(0, a, pts);
   p = pi*rho/a;
   [xGrid, yGrid] = meshgrid(xi, yi);
   %redundancy checks ( xi and yi should already be removed if to large
```

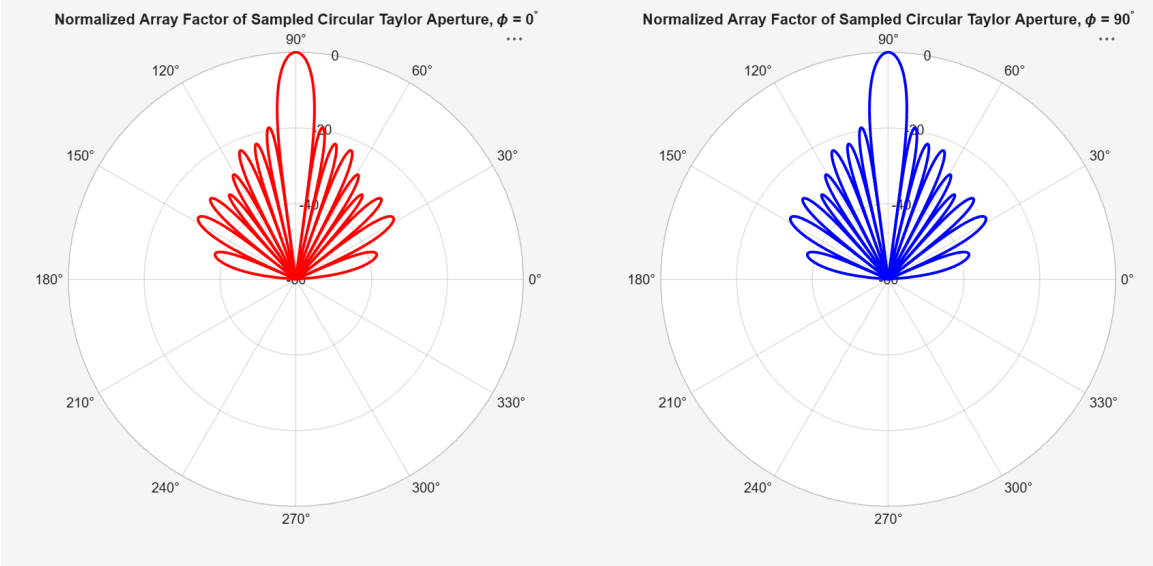
```
  pmn = sqrt( (xGrid).^2 + (yGrid).^2);
  %continuous aperture.
  g_rho = Taylor_samp(gamma,un,nbar, p);
  g = g_rho./max(g_rho(:));
  %map it to 2D circle
  %discrete rectangular grid
  w = zeros(size(xGrid));
  p_dis = pmn ./ a * pi;
  w_dis = Taylor_samp(gamma, un, nbar, p_dis);
  %remove elements outside of aperture
  w(sqrt( (abs(xGrid)+d/2).^2 + (abs(yGrid) + d/2).^2)>5 ) = 0;
  w = w_dis./max(w_dis(:));
end
function [w] = Taylor_samp(gamma,un, nbar, p)
  %p <= pi
  %S0 = 1/2
  sum = 0.5 * ones(size(p));
  for i = 1:nbar-1
     num = 1;
     den = 1;
     for m = 1:nbar-1
        num = num * (1 - (gamma(i)/un(m))^2);
        if i ~= m
           den = den * (1 - (gamma(i)/gamma(m))^2);
        end
     end
     % Finite limit at each original positive null
     S = -0.5 * besselj(0, pi*gamma(i)) * num/den;
     frac = S / besselj(0, pi*gamma(i))^2;
     sum = sum + frac * besselj(0, gamma(i)*p);
  end
  w = 2/pi^2 * sum;
end
function [AF, AF_dB, AFphase] = compAF_UAplanar(pts, Nx, Ny, w,d)
  %w should have size mxm
  %coordinate transform
  u = linspace(-1,1, pts);
  v = linspace(-1,1, pts);
  [uGrid, vGrid] = meshgrid(u,v);
  xi = (-(Nx-1)/2:(Nx-1)/2) * d;
  yi = (-(Ny-1)/2:(Ny-1)/2) * d;
  AF = zeros(pts,pts);
  for n = 1:Nx
     for m = 1:Ny
        AF = AF + w(n,m) *exp(1i * 2 * pi * (xi(n) * ...
           uGrid + yi(m) * vGrid ) );
     end
  end
  AF_dB = 20.* log10(abs(AF));
  AFphase = angle(AF);
end
```

Part 5 Sampled Taylor Aperture Pattern

The plotted sampled pattern has a broadside maximum and similar principal-plane cuts. Its outer sidelobes are less circular than those in the continuous result. Finite sampling can change the sidelobe structure even when the overall aperture shape is retained.







MATLAB Code

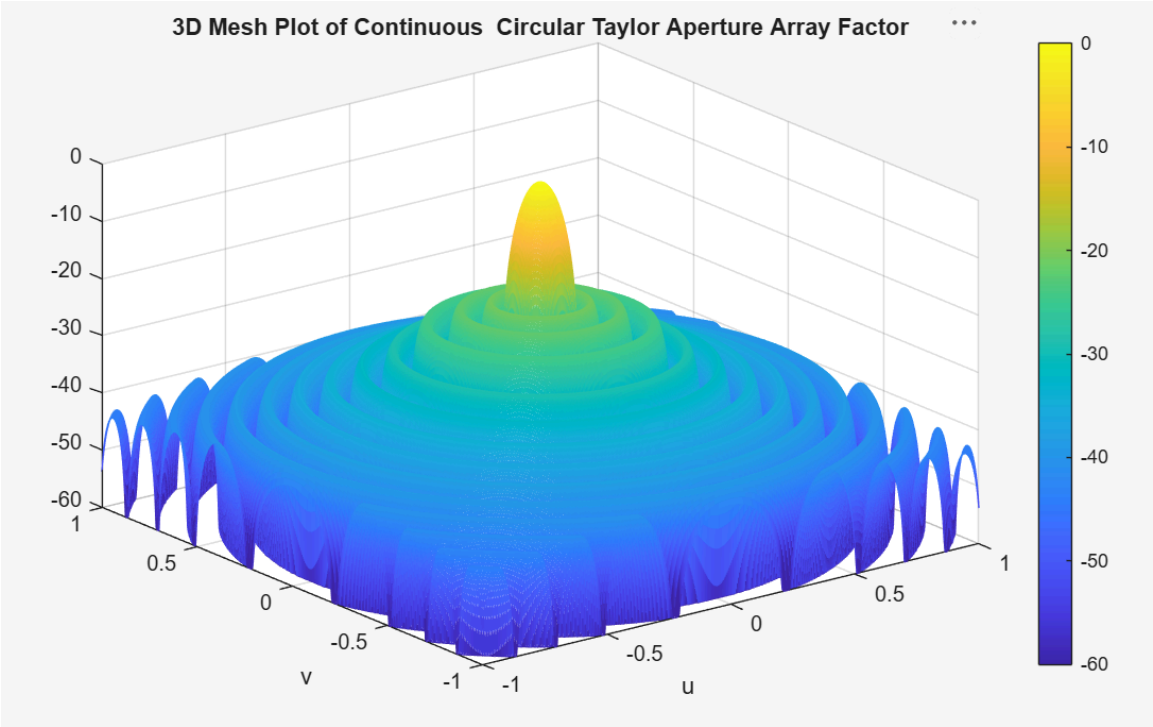
```
clc;
clear all;
%params
a = 5; %in lambda
d = 0.5;
pts = 1.001e3;
SLL = 20;
nbar = 4;
%calc un  and gamma
j1 = @(x) besselj(1, x);
mu = zeros(1, nbar);
for i = 1:nbar
   interval = [i*pi, (i + 0.5)*pi];
   mu(i) = fzero(j1, interval);
end
gamma = mu/pi;
A = acosh(10^(SLL/20))/pi;
% New null locations in lecture notation
n = 1:nbar-1;
un = gamma(nbar) * sqrt( ...
   (A^2 + (n - 0.5).^2) / ...
   (A^2 + (nbar - 0.5)^2));
%Plot pattern of continuous Taylor Aperture.
rho = linspace(1,a, pts);
p_cont = pi * rho/a;
pts2 = 501;
uCont = linspace(-1, 1, pts2);
vCont = linspace(-1, 1, pts2);
[uContGrid, vContGrid] = meshgrid(uCont, vCont);
U = 2*a*sqrt(uContGrid.^2 + vContGrid.^2);
g = @(p) Taylor_samp(gamma,un,nbar, p);
Fint = @(p) p.* g(p) .* besselj(0, U .* p);
```

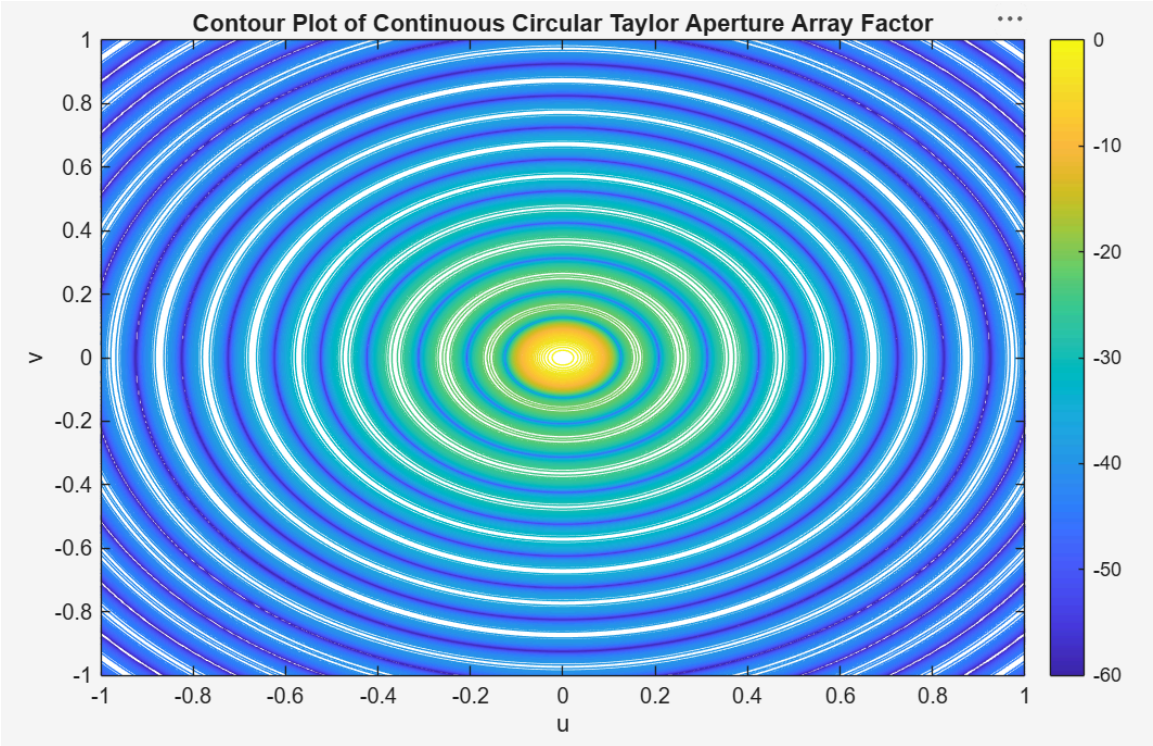
```
F = integral(Fint, 0, pi, 'ArrayValued',true);
F_dB = 20 * log10(abs(F));
F_dBnorm = F_dB - max(F_dB(:));
F_dBnorm(F_dBnorm < -60)= -60;
%now to plot
c = F_dBnorm;
c(c<-60) = NaN;
figure(1);
clf;
mesh(uCont,vCont,c);
colorbar;
clim([-60 0]);
xlabel('u');
ylabel('v');
title('3D Mesh Plot of Continuous  Circular Taylor Aperture Array Factor');
figure(2);
contour(uCont, vCont, c,100);
colorbar;
clim([-60 0]);
xlabel('u');
ylabel('v');
title('Contour Plot of Continuous Circular Taylor Aperture Array Factor');
%2D plane cuts;
u0 = find(uCont == 0);
v0 = find(vCont == 0);
theta = acos(uCont);
figure(3);
tiledlayout(1,2);
nexttile;
rho = F_dBnorm (u0, :);
h = polarplot(theta,rho+60, 'LineWidth',2, 'color','r');
haxes = get(h, 'Parent');
haxes.RTickLabel = {'-60','-40','-20','0'};
title('Normalized Array Factor of Continuous Circular Taylor Aperture, \phi = 0^\circ');
nexttile;
rho = F_dBnorm (:, v0);
h = polarplot(theta,rho+60, 'LineWidth',2, 'color','b');
haxes = get(h, 'Parent');
haxes.RTickLabel = {'-60','-40','-20','0'};
title('Normalized Array Factor of Continuous Circlar Taylor Aperture, \phi = 90^\circ');
%FUNCTIONS
function [g, w] = circ_TDw(a,gamma,un, nbar, SLL, xi, yi,d)
   pts = 1e4;
   rho = linspace(0, a, pts);
   p = pi*rho/a;
   [xGrid, yGrid] = meshgrid(xi, yi);
   pmn = sqrt( (xGrid).^2 + (yGrid).^2);
   %continuous aperture.
   g_rho = Taylor_samp(gamma,un,nbar, p);
   g = g_rho./max(g_rho(:));
   %map it to 2D circle
```

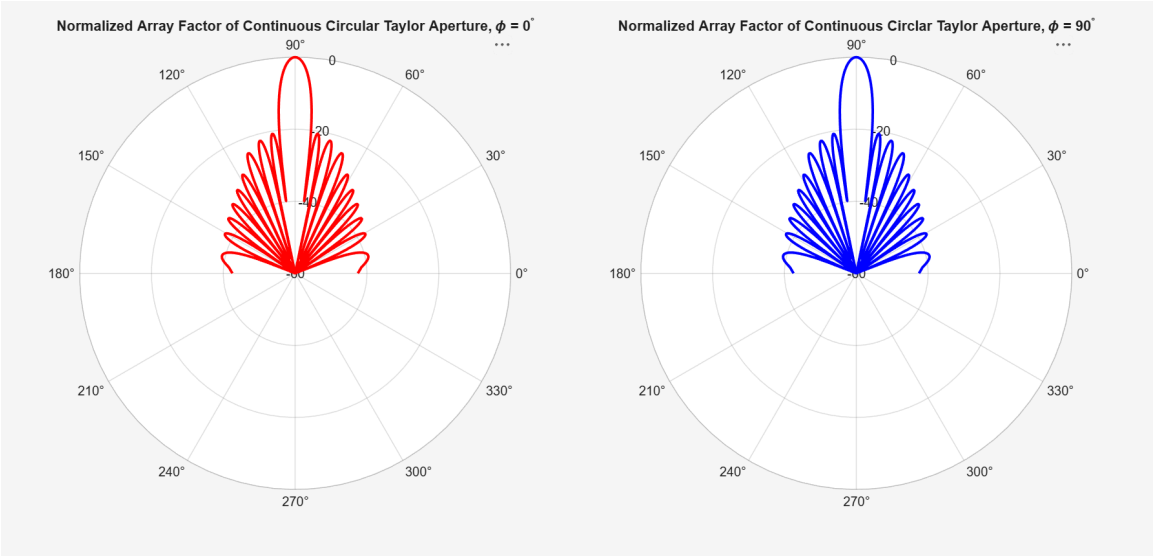
```
  %discrete rectangular grid
  w = zeros(size(xGrid));
  p_dis = pmn ./ a * pi;
  w_dis = Taylor_samp(gamma, un, nbar, p_dis);
  %remove elements outside of aperture
  w_dis(sqrt( (abs(xGrid)+d/2).^2 + (abs(yGrid) + d/2).^2)>5 ) = 0;
  w = w_dis./max(w_dis(:));
end
function [w] = Taylor_samp(gamma,un, nbar, p)
  %p <= pi
  %S0 = 1/2
  sum = 0.5 * ones(size(p));
  for i = 1:nbar-1
     num = 1;
     den = 1;
     for m = 1:nbar-1
        num = num * (1 - (gamma(i)/un(m))^2);
        if i ~= m
           den = den * (1 - (gamma(i)/gamma(m))^2);
        end
     end
     % Finite limit at each original positive null
     S = -0.5 * besselj(0, pi*gamma(i)) * num/den;
     frac = S / besselj(0, pi*gamma(i))^2;
     sum = sum + frac * besselj(0, gamma(i)*p);
  end
  w = 2/pi^2 * sum;
end
function [AF, AF_dB, AFphase] = compAF_UAplanar(pts, Nx, Ny, w,d)
  %w should have size mxm
  %coordinate transform
  u = linspace(-1,1, pts);
  v = linspace(-1,1, pts);
  [uGrid, vGrid] = meshgrid(u,v);
  xi = (-(Nx-1)/2:(Nx-1)/2) * d;
  yi = (-(Ny-1)/2:(Ny-1)/2) * d;
  AF = zeros(pts,pts);
  for n = 1:Nx
     for m = 1:Ny
        AF = AF + w(n,m) *exp(1i * 2 * pi * (xi(n) * ...
           uGrid + yi(m) * vGrid ) );
     end
  end
  AF_dB = 20.* log10(abs(AF));
  AFphase = angle(AF);
end
```

Continuous Taylor Aperture Pattern

The continuous calculation integrates the radial distribution over the full aperture. Its contour plot forms smooth concentric rings, and the principal-plane cuts have the same shape. This follows from the rotational symmetry of the distribution.







MATLAB Code

```
clc;
clear all;
%params
a = 5; %in lambda
d = 0.5;
pts = 1.001e3;
SLL = 20;
nbar = 4;
%calc un  and gamma
j1 = @(x) besselj(1, x);
mu = zeros(1, nbar);
for i = 1:nbar
   interval = [i*pi, (i + 0.5)*pi];
   mu(i) = fzero(j1, interval);
end
gamma = mu/pi;
A = acosh(10^(SLL/20))/pi;
% New null locations in lecture notation
n = 1:nbar-1;
un = gamma(nbar) * sqrt( ...
   (A^2 + (n - 0.5).^2) / ...
   (A^2 + (nbar - 0.5)^2));
%Plot pattern of continuous Taylor Aperture.
rho = linspace(1,a, pts);
p_cont = pi * rho/a;
pts2 = 501;
uCont = linspace(-1, 1, pts2);
vCont = linspace(-1, 1, pts2);
[uContGrid, vContGrid] = meshgrid(uCont, vCont);
U = 2*a*sqrt(uContGrid.^2 + vContGrid.^2);
g = @(p) Taylor_samp(gamma,un,nbar, p);
Fint = @(p) p.* g(p) .* besselj(0, U .* p);
```

```
F = integral(Fint, 0, pi, 'ArrayValued',true);
F_dB = 20 * log10(abs(F));
F_dBnorm = F_dB - max(F_dB(:));
F_dBnorm(F_dBnorm < -60)= -60;
%now to plot
c = F_dBnorm;
c(c<-60) = NaN;
figure(1);
clf;
mesh(uCont,vCont,c);
colorbar;
clim([-60 0]);
xlabel('u');
ylabel('v');
title('3D Mesh Plot of Continuous  Circular Taylor Aperture Array Factor');
figure(2);
contour(uCont, vCont, c,100);
colorbar;
clim([-60 0]);
xlabel('u');
ylabel('v');
title('Contour Plot of Continuous Circular Taylor Aperture Array Factor');
%2D plane cuts;
u0 = find(uCont == 0);
v0 = find(vCont == 0);
theta = acos(uCont);
figure(3);
tiledlayout(1,2);
nexttile;
rho = F_dBnorm (u0, :);
h = polarplot(theta,rho+60, 'LineWidth',2, 'color','r');
haxes = get(h, 'Parent');
haxes.RTickLabel = {'-60','-40','-20','0'};
title('Normalized Array Factor of Continuous Circular Taylor Aperture, \phi = 0^\circ');
nexttile;
rho = F_dBnorm (:, v0);
h = polarplot(theta,rho+60, 'LineWidth',2, 'color','b');
haxes = get(h, 'Parent');
haxes.RTickLabel = {'-60','-40','-20','0'};
title('Normalized Array Factor of Continuous Circlar Taylor Aperture, \phi = 90^\circ');
%FUNCTIONS
function [g, w] = circ_TDw(a,gamma,un, nbar, SLL, xi, yi,d)
   pts = 1e4;
   rho = linspace(0, a, pts);
   p = pi*rho/a;
   [xGrid, yGrid] = meshgrid(xi, yi);
   pmn = sqrt( (xGrid).^2 + (yGrid).^2);
   %continuous aperture.
   g_rho = Taylor_samp(gamma,un,nbar, p);
   g = g_rho./max(g_rho(:));
   %map it to 2D circle
```

```
  %discrete rectangular grid
  w = zeros(size(xGrid));
  p_dis = pmn ./ a * pi;
  w_dis = Taylor_samp(gamma, un, nbar, p_dis);
  %remove elements outside of aperture
  w_dis(sqrt( (abs(xGrid)+d/2).^2 + (abs(yGrid) + d/2).^2)>5 ) = 0;
  w = w_dis./max(w_dis(:));
end
function [w] = Taylor_samp(gamma,un, nbar, p)
  %p <= pi
  %S0 = 1/2
  sum = 0.5 * ones(size(p));
  for i = 1:nbar-1
     num = 1;
     den = 1;
     for m = 1:nbar-1
        num = num * (1 - (gamma(i)/un(m))^2);
        if i ~= m
           den = den * (1 - (gamma(i)/gamma(m))^2);
        end
     end
     % Finite limit at each original positive null
     S = -0.5 * besselj(0, pi*gamma(i)) * num/den;
     frac = S / besselj(0, pi*gamma(i))^2;
     sum = sum + frac * besselj(0, gamma(i)*p);
  end
  w = 2/pi^2 * sum;
end
function [AF, AF_dB, AFphase] = compAF_UAplanar(pts, Nx, Ny, w,d)
  %w should have size mxm
  %coordinate transform
  u = linspace(-1,1, pts);
  v = linspace(-1,1, pts);
  [uGrid, vGrid] = meshgrid(u,v);
  xi = (-(Nx-1)/2:(Nx-1)/2) * d;
  yi = (-(Ny-1)/2:(Ny-1)/2) * d;
  AF = zeros(pts,pts);
  for n = 1:Nx
     for m = 1:Ny
        AF = AF + w(n,m) *exp(1i * 2 * pi * (xi(n) * ...
           uGrid + yi(m) * vGrid ) );
     end
  end
  AF_dB = 20.* log10(abs(AF));
  AFphase = angle(AF);
end
```

Part 2: Array Design, Modelling, and Simulation

Air Filled Coaxial Feed

A test bench was first made for the air-core coax. PEC was used for the inner probe, with a Perfect E boundary on the outer surface of the air region. Wave ports were applied to both ends to determine the characteristic impedance and S-parameters.

The lossless coax relation gives an outer conductor inner radius of approximately 1.38 mm for the specified inner pin and impedance.

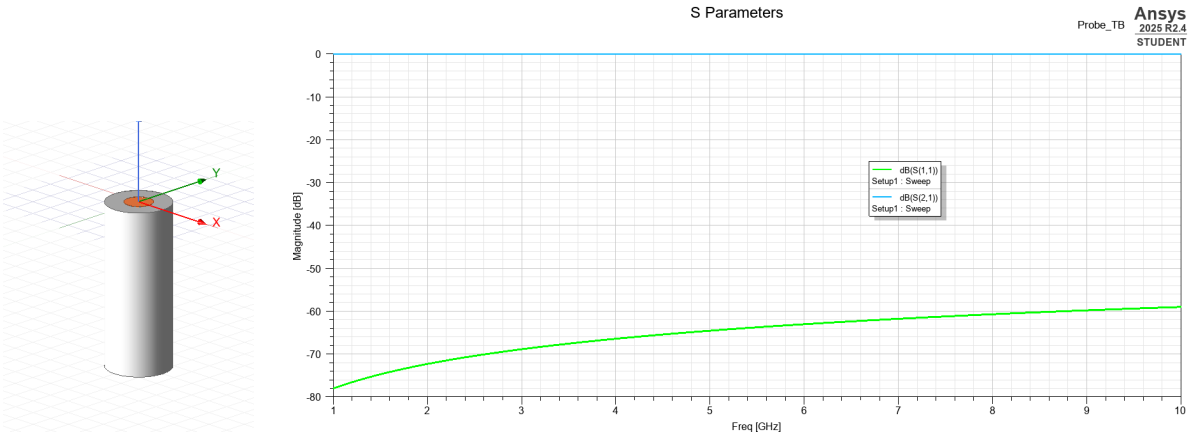


Figure 11. Air-filled coaxial test geometry (left) and simulated reflection and transmission (right).

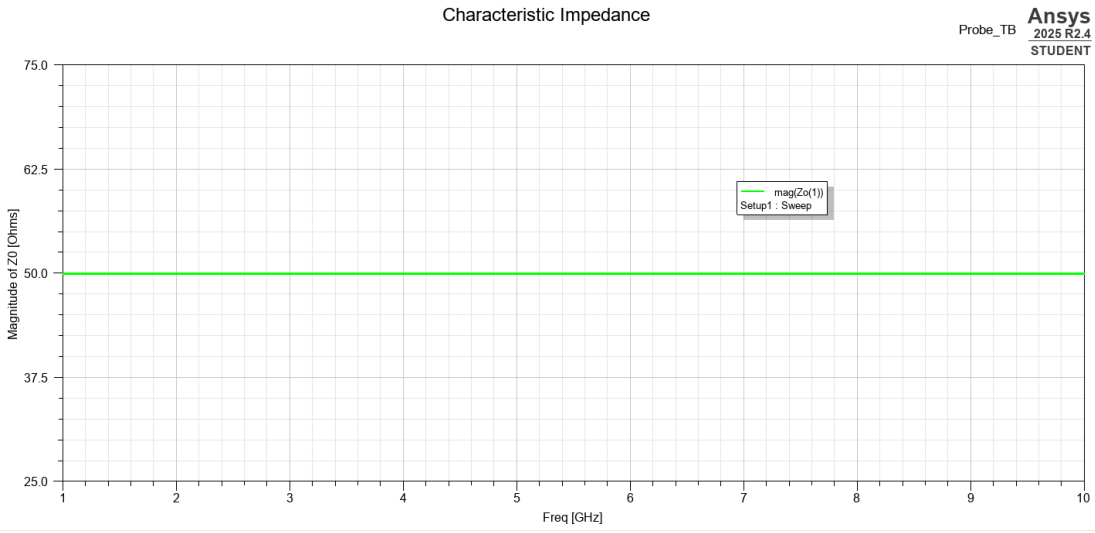


Figure 12. Simulated coaxial characteristic impedance.

The simulated S-parameters and characteristic impedance show that the coax geometry serves as a valid feed for the patch antenna.

The initial patch dimensions were calculated using the MATLAB script included at the end. The probe was offset along the resonant length to improve matching, and the patch corners were rounded as a tuning adjustment.

Rogers Duroid 5880 is used for the substrate, with approximately half-wavelength unit-cell spacing. Lattice pairs and Floquet ports are used to model the infinite array.

Figure 13. Initial probe-fed patch geometry and surrounding simulation region.

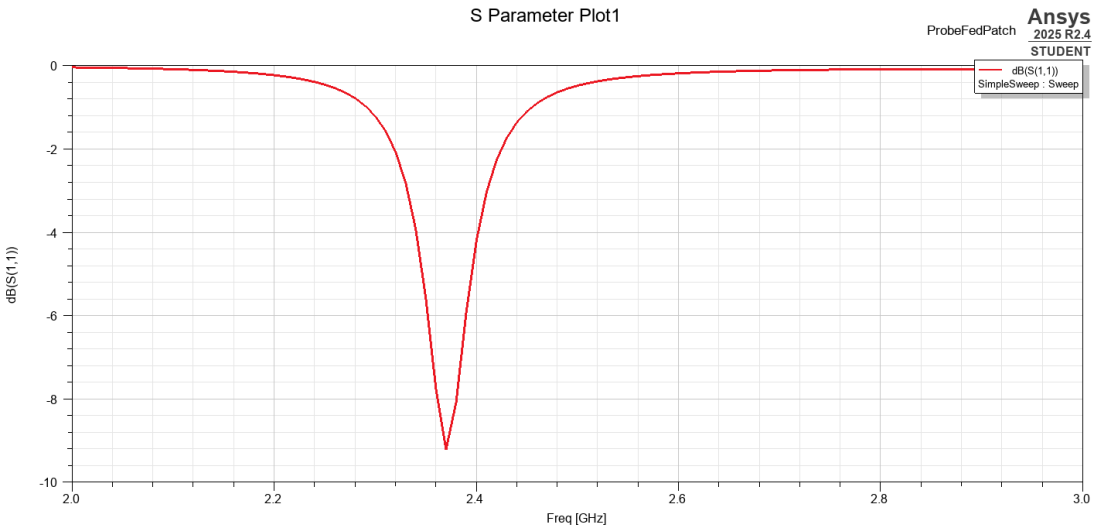


Figure 14. Initial input matching of the probe-fed patch before optimization.

The initial response is close to the target based on analytical calculations, but matching is poor. Genetic algorithm optimization is used to achieve the desired characteristics.

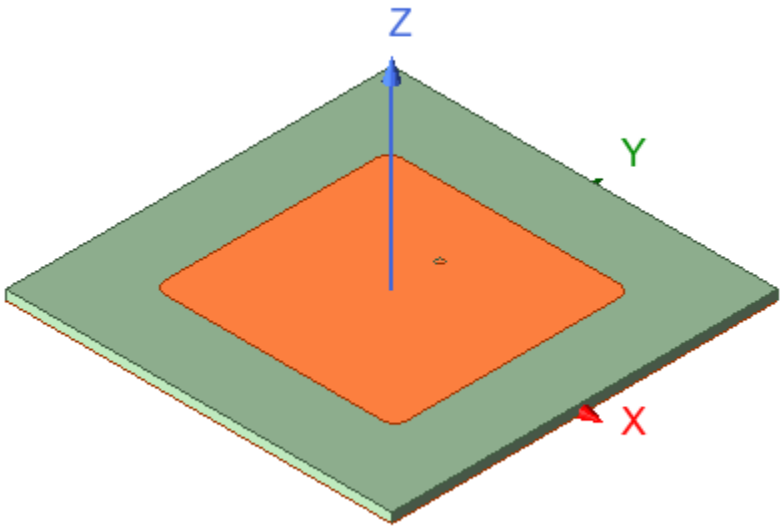


Figure 15. Optimized patch geometry.

The geometry shown above is produced after brief optimization. It is slightly rectangular, which may result in different scanning performance along different planes.

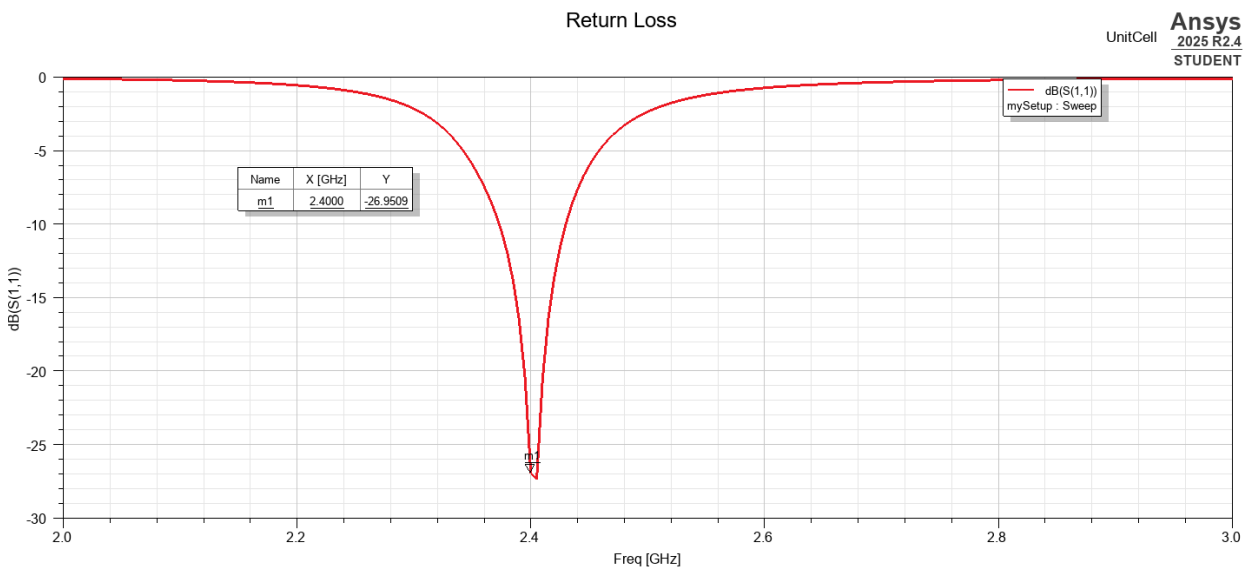


Figure 16. Optimized input matching of the patch.

The optimized design shows the desired input match at the frequency of interest.

Radiation Pattern

The radiation patterns were obtained with coax excitation. Both plane cuts show broadside radiation, with little difference in the main beam. The three-dimensional pattern also shows a broad main lobe above the patch.

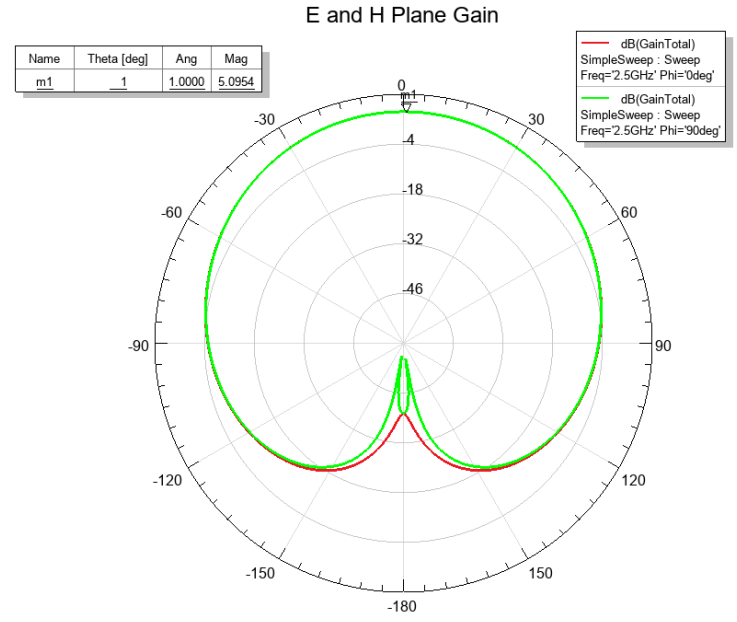


Figure 17. Total-gain plane cuts of the patch with coax excitation.

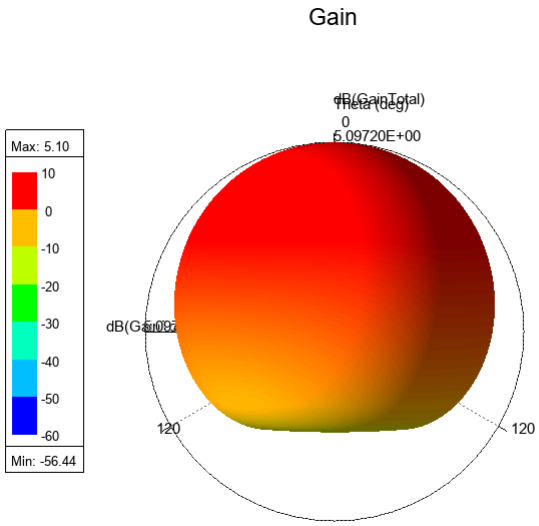


Figure 18. Three-dimensional total-gain pattern with coax excitation.

Field Analysis

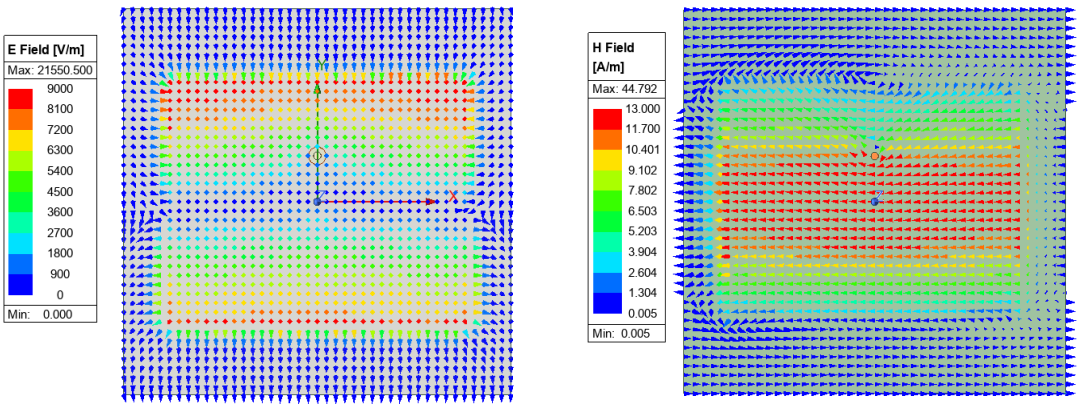


Figure 19. Electric field (left) and magnetic field (right) in the substrate.

The electric field is strongest near the radiating edges and weaker near the center. The magnetic field is strongest through the middle of the patch. These distributions are consistent with TM_{10} operation.

Array Modelling and Simulation

After optimization of the unit cell element, PyAEDT was leveraged to construct the array topology and amplitude distribution computed in part 1. HFSS FADDM4 hybrid array solver was used to truncate the simulation time and circumvent the mesh limit imposed by the Ansys Electronic Desktop licensure. The full array model is shown below.

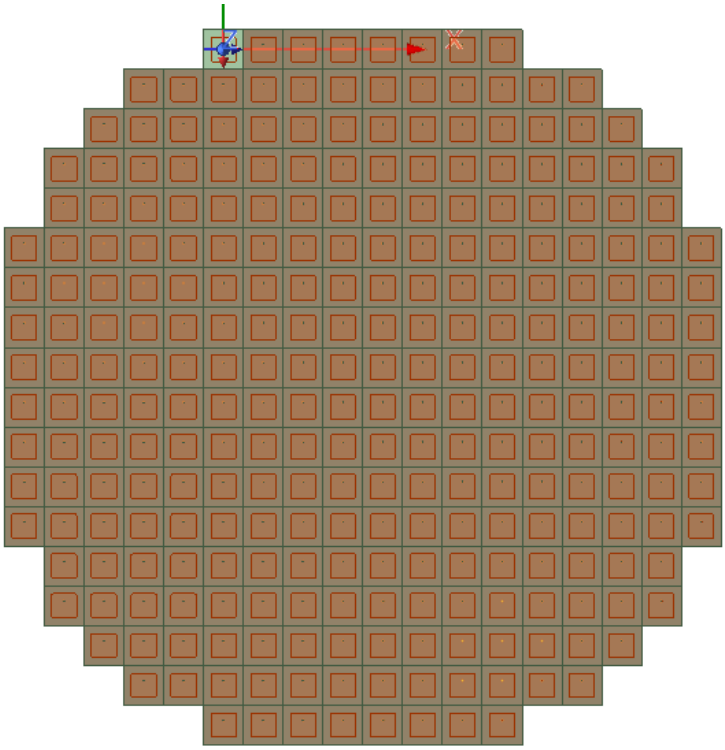


Figure 20: HFSS array model.

Currently, I am still battling with my computer's memory. I simply do not have enough compute to run this simulation.

MATLAB Code for Initial Patch Dimensions

The MATLAB script below was used to calculate the initial patch dimensions. Only the patch width and length were used for the probe-fed design.

```
clear;
clc;

f_GHz = input('Resonant frequency (GHz): ');
er = input('Relative permittivity: ');
h_mm = input('Substrate height (mm): ');

c = 3e8;
f = f_GHz*1e9;
h = h_mm*1e-3;
Z0 = 50;

% Patch dimensions
W = c/(2*f)*sqrt(2/(er + 1));
er_eff = (er + 1)/2 + (er - 1)/2*(1 + 12*h/W)^(-0.5);

dL = 0.412*h*((er_eff + 0.3)*(W/h + 0.264))/...
   ((er_eff - 0.258)*(W/h + 0.8));

L_eff = c/(2*f*sqrt(er_eff));
L = L_eff - 2*dL;

% Edge impedance and bandwidth
Z_edge = 90*(er^2/(er - 1))*(L/W)^2;
BW = f^2*3.77*((er - 1)/er^2)*(W*h/(c*L));

% 50-ohm microstrip feed width
A = Z0/60*sqrt((er + 1)/2) + ...
   (er - 1)/(er + 1)*(0.23 + 0.11/er);
W_h = 8*exp(A)/(exp(2*A) - 2);

if W_h > 2
   B = 60*pi^2/(Z0*sqrt(er));
   W_h = 2/pi*(B - 1 - log(2*B - 1) + ...
      (er - 1)/(2*er)*(log(B - 1) + 0.39 - 0.61/er));
end

W0 = W_h*h;

% Inset dimensions
D = L/pi*acos((Z0/Z_edge)^(1/4));
S = 1.5*W0;
notch_width = W0 + 2*S;

fprintf('\nInset-fed patch dimensions\n');
fprintf('--------------------------\n');
fprintf('Patch width, W:          %.4f mm\n', W*1e3);
fprintf('Patch length, L:         %.4f mm\n', L*1e3);
fprintf('Feedline width, W0:      %.4f mm\n', W0*1e3);
fprintf('Inset depth, D:          %.4f mm\n', D*1e3);
fprintf('Inset side gap, S:       %.4f mm\n', S*1e3);
fprintf('Total notch width:       %.4f mm\n', notch_width*1e3);
fprintf('Effective permittivity:  %.4f\n', er_eff);
```

```
fprintf('Patch edge impedance:    %.2f ohms\n', Z_edge);
fprintf('Estimated bandwidth:     %.2f MHz\n', BW/1e6);

fprintf('\nS is an initial value and should be tuned in an EM simulation.\n');
```